RFA Spec: Configuration File I/O Toolbox

Project: Configuration File I/O Toolbox **Author:** Michelle Hirsch **Date:** January 15, 2026 **Revised:** February 2026 **Status:** Draft

Project Details

Project Kickoff

Motivation and Key Users

Problem Statement

MATLAB lacks native support for reading and writing modern configuration file formats that are ubiquitous in software development:

- **YAML** — Used by GitHub Actions, Docker Compose, Kubernetes, Ansible, ML experiment frameworks (Hydra, W&B), and countless other tools
- **TOML** — Used by Python (pyproject.toml), Rust (Cargo.toml), and modern configuration systems

MATLAB users working in mixed-language environments or DevOps workflows must either:

1. Use external tools to convert files to JSON/XML (which MATLAB does support)
2. Write custom parsers for each project
3. Use third-party File Exchange submissions of varying quality

Key Users

User Type	Description
MATLAB Developers	Engineers building applications that need configuration management
Platform Engineers	Teams managing CI/CD pipelines with MATLAB components
Package Maintainers	Developers creating MATLAB toolboxes with modern metadata
ML/AI Researchers	Data scientists using YAML for experiment hyperparameter configuration
DevOps/MATLAB Engineers	Platform engineers managing MATLAB server deployments on Kubernetes

Workflows Supported

1. **CI/CD Pipeline Management** — Read and modify GitHub Actions workflows programmatically
2. **Project Configuration** — Read and modify pyproject.toml/future matlab.toml for Python/MATLAB interop projects

3. **Container Orchestration** — Manage Docker Compose and Kubernetes configurations
4. **Application Settings** — Store and retrieve application configuration with structured data
5. **ML Experiment Management** — Load and save hyperparameter configurations for reproducible experiments

Technical Constraints

- **MATLAB Version:** R2022b or later (required for `dictionary` type with value semantics)
- **No External Dependencies:** Pure MATLAB implementation, no Java libraries or MEX files
- **Subset Parsers:** Focus on common patterns rather than full spec compliance

Risks and Assumptions

Risk	Mitigation
Full YAML 1.2 spec is extremely complex	Implement subset covering 95%+ of real-world config files
Handle vs value semantics confusion	Value classes for intuitive MATLAB behavior
Performance on large files	Target config files (<1MB), not data files
Array/scalar round-trip fidelity	Document `SequenceRule` tradeoffs clearly; provide `"cell"` option for strict round-trip

Format Background

Configuration files store settings that control how software behaves. Unlike data files — which hold measurements, results, or records — config files are typically written and maintained by humans, and are read by software at startup or runtime. The format a project chooses reflects how it expects users to interact with those settings.

Several formats are in common use:

JSON and XML are general-purpose serialization formats designed primarily for computer-to-computer communication. They are machine-friendly: strict syntax, no ambiguity, and fast to parse. JSON intentionally omits comment support — it was designed as a data interchange format where comments have no role. Both formats work as config files, but they aren't very pleasant to read or write.

TOML and YAML were designed specifically for humans authoring config files. Quoting requirements are minimal. Structure reads naturally. Neither format is intended for high-volume data exchange — they optimize for clarity and editability over parsing performance. These are the formats this toolbox targets.

Regardless of syntax, both formats represent the same fundamental structure: **a hierarchy of named values**. A MATLAB struct is the right mental model. The top level is like a struct with named fields; each field holds a scalar value (a number, a string, or a boolean), an array, or a nested struct. Reading a config file loads that hierarchy from text; writing one serializes it back.

```
% Think of a config file like a struct
config.debug            = false;                          % Scalar value
config.releases         = ["R2022b", "R2023a", "R2024a"];   % Array
config.database.host    = "localhost";                    % Nested
struct
config.database.port    = 5432;
```

The interesting challenges arise at the edges of this mapping: YAML and TOML have naming conventions for their keys that don't align with MATLAB's identifier rules, and unlike MATLAB, they distinguish between scalars and arrays.

YAML

YAML (YAML Ain't Markup Language) is an indentation-based format widely used for CI/CD pipelines (GitHub Actions, GitLab CI), container orchestration (Docker Compose, Kubernetes), ML experiment frameworks (Hydra, Weights & Biases), and many other tools.

Core concepts

Key-value pairs

```
name: my-project
version: 1.0
debug: false
```

Most strings do not require quotes. Type is inferred from the value's appearance, e.g. `true`/`false` are Booleans while `"true"`/`"false"` are strings. 1.0 is a float, 5432 is an integer

Mappings — nested structure:

```
database:
  host: localhost
  port: 5432
  name: mydb
```

Indentation defines nesting — `host`, `port`, and `name` are keys within `database`. There are no braces or explicit delimiters; whitespace is the structure. Indent any number of spaces (not tabs), just be consistent within a given block.

Sequences — lists (conceptually like 1-D arrays). YAML supports two styles: **Block** and **Flow**

```
# Block style — each item on its own line, readable
releases:
  - R2022b
```

```
  - R2023a
  - R2024a

# Flow style — compact inline
releases: [R2022b, R2023a, R2024a]
```

Both are valid YAML and semantically identical. The choice is stylistic. Block is more readable for longer lists; flow is compact for short ones. The values in a sequence can be mixed types; no requirement for homogeneity.

Comments:

```
# Database configuration
database:
  host: localhost  # override in production
```

Comments run to the end of the line. They are typically discarded by parsers and not preserved through a round-trip.

Real-world example — GitHub Actions workflow fragment

```
on:                                    # on is a mapping with key push
  push:                                # push is a mapping with key
branches
    branches: [main]                   # branches is a sequence of
string scalars (flow style)

jobs:
  test:
    runs-on: ubuntu-latest             # runs-on is a scalar string.
    strategy:
      matrix:
        matlab: [R2022b, R2023a, R2024a]  # matlab is a sequence of string
scalars
    steps:                             # steps is a sequence of
mappings, each with keys 'name' and 'uses'
      - name: Set up MATLAB            # '-' begins a new mapping in
the sequence
        uses: matlab-actions/setup-matlab@v2
      - name: Run tests                # '-' begins the second mapping
        uses: matlab-actions/run-tests@v2
```

Summary of YAML details we'll need to consider

Key names with hyphens and special characters. `runs-on` is a valid YAML key, but it is not a valid MATLAB variable name — the hyphen is syntactically ambiguous with subtraction. This is ubiquitous in real

YAML: GitHub Actions uses `runs-on`, `pull-request`, `workflow-dispatch`; HTTP headers use `x-custom-header`, `content-type`.

Scalar vs. single-element sequence. YAML distinguishes `branches: main` (a scalar string) from `branches: [main]` (a one-element sequence). In MATLAB, `"main"` and `["main"]` are identical — there is no native way to represent this distinction. Users will need a way to explicitly distinguish.

Indentation. YAML uses indentation as syntax, not style. Two spaces is the most common convention; four spaces is also widely used. The exact amount is not specified by the format — any consistent indentation is valid. Users may want flexibility.

Section spacing. Blank lines between top-level sections significantly improve readability in longer config files. This is a common convention but not required by the format. Users may want flexibility to help write files that look familiar.

Numeric precision. Floating-point values can be written with varying precision: `1.2345678` vs `1.23`. The right choice depends on context — a measurement that must round-trip exactly needs full precision; a threshold set by a human benefits from a clean representation.

TOML

TOML (Tom's Obvious, Minimal Language) was created by Tom Preston-Werner (GitHub co-founder) as a config format with unambiguous, explicitly typed values. It is used by `pyproject.toml` (Python project metadata), `Cargo.toml` (Rust packages), and many modern configuration systems. It will be used by `matlab.toml`, the forthcoming MATLAB project definition file.

Core concepts

Key-value pairs

```
name = "my-project"
version = "1.0.0"
debug = false
port = 5432
pi = 3.14159
```

Strings must be quoted. Booleans are `true`/`false`. Integers and floats are unquoted. Unlike YAML, strings are always quoted.

Tables — nested structure. TOML supports two syntaxes **expanded** and **inline**:

```
# Expanded: [section] header, then key-value pairs below
[database]
host = "localhost"
port = 5432

# Inline: compact, written on one line
database = {host = "localhost", port = 5432}
```

Both are semantically equivalent. Expanded tables are the standard for top-level configuration sections; inline tables are used when the nested data is simple enough to read comfortably on one line.

Arrays — TOML supports both single line and multiline arrays:

```
# Single line — the standard style for most TOML arrays
releases = ["R2022b", "R2023a", "R2024a"]

# Multiline — for longer arrays
releases = [
  "R2022b",
  "R2023a",
  "R2024a",
]
```

The values in a sequence can be mixed types; no requirement for homogeneity.

Array of tables — repeated structured records, two syntaxes:

```
# Expanded [[double-bracket]] syntax — the standard, most readable
[[steps]]
name = "Set up MATLAB"
uses = "matlab-actions/setup-matlab@v2"

[[steps]]
name = "Run tests"
uses = "matlab-actions/run-tests@v2"

# Inline array syntax — compact alternative
steps = [
  {name = "Set up MATLAB", uses = "matlab-actions/setup-matlab@v2"},
  {name = "Run tests", uses = "matlab-actions/run-tests@v2"},
]
```

`[[double brackets]]` defines an array of tables — a list of records, each with the same set of keys. This is TOML's equivalent of what YAML expresses as a sequence of mappings. The expanded form is by far the most common in real TOML files.

String types — TOML has four:

```
# Basic string — supports escape sequences (\n, \t, \\ etc.)
message = "Hello\nWorld"
path = "C:\\Users\\name\\Documents"   # each backslash must be doubled

# Literal string — no escape processing; content taken verbatim
path = 'C:\Users\name\Documents'      # backslashes are literal
```

```toml
# Multiline basic string — for long text
description = """
This spans
multiple lines.
"""

# Multiline literal string — verbatim, multi-line
template = '''
{{variable}} syntax is not processed here
\n is two characters, not a newline
'''
```

The key practical distinction: **literal strings (single quotes) are the natural choice for Windows paths** because every \ in a basic string must be written as \\. For most config values this doesn't arise; it matters whenever the value contains backslashes.

Comments:

```toml
# Project metadata
[project]
name = "my-project"  # PEP 518 required field
```

Real-world example — pyproject.toml fragment

```toml
[build-system]                          # build-system is a table with
keys requires and build-backend
requires = ["hatchling"]                # requires is an array of
strings
build-backend = "hatchling.build"       # build-backend is a string
scalar

[project]                               # project is a table with 3
keys
name = "my-package"
version = "0.1.0"                       # string scalar — quoted, not a
number
dependencies = ["numpy>=1.21", "scipy"] # dependencies is an array of
strings

[[project.authors]]                     # project.authors is an array
of tables
name = "Jane Smith"
email = "jane@example.com"

[[project.authors]]                     # [[...]] begins the second
table in the array
name = "John Doe"
email = "john@example.com"
```

Summary of TOML details we'll need to consider

Key and table names with hyphens. `[build-system]` appears in essentially every `pyproject.toml`. Same challenge as YAML: `build-system` is not a valid MATLAB identifier.

Multiple syntax options for the same structure. Tables, arrays, and arrays of tables each have two syntactically distinct but semantically equivalent forms — expanded vs. inline. When generating TOML from MATLAB, choices must be made about which form to produce, and different users will have different preferences depending on context.

YAML and TOML Summary

Both formats represent the same underlying structure — a hierarchy of named values — but use different terminology and syntax conventions. This table maps the core concepts across the two formats and into MATLAB.

Concepts

Concept	YAML term	TOML term	MATLAB equivalent
Named element	**Key**	**Key**	Field (struct), Key (dict), Property (class)
Ordered collection of key-value pairs	**Mapping**	**Table**	struct
Ordered list of values	**Sequence**	**Array**	array or cell array
Atomic value (number, string, boolean)	**Scalar**	**Value**	scalar (1-element array)

Syntax

Concept	YAML term	TOML term	Closest MATLAB equivalent
Comments	`# text`	`# text`	`% text`
String quoting	Optional — most values unquoted	Required for all strings	Required (`"..."` or `'...'`)
Nesting syntax	Indentation	`[section]` header; dotted keys (`a.b`)	`.` dot notation
Inline nested structure	**Flow mapping**: `{key: val}`	**Inline table**: `{key = val}`	`s.key = val`
Inline list	**Flow sequence**: `[a, b, c]`	Array: `["a", "b", "c"]`	`[a b c]` or `{"a","b","c"}`

Concept	YAML term	TOML term	Closest MATLAB equivalent
List of structured records (see below)	**Sequence** of **mappings**	**Array** of **tables**	struct array

YAML - Sequence of Mappings

```yaml
# YAML — sequence of mappings (block style)
steps:
  - name: build
    cmd: make
  - name: test
    cmd: make test
```

TOML - Array of Tables

```toml
# TOML — array of tables
[[steps]]
name = "build"
cmd = "make"

[[steps]]
name = "test"
cmd = "make test"
```

MATLAB - Struct Array

```matlab
% MATLAB — struct array
steps(1).name = "build";  steps(1).cmd = "make";
steps(2).name = "test";   steps(2).cmd = "make test";
```

Valid key/field names

Rule	YAML	TOML	MATLAB
Allowed characters	Most printable chars except space	`A–Za–z`, `0–9`, `–`, `_`	`A–Za–z`, `0–9`, `_`
Names with spaces	`"key with spaces"`	`"key with spaces"`	Not supported
Must start with letter	✗	✗	✓

Requirements Analysis

User Roles and Goals

ID	Priority	User Role	User Goal	Notes
RG_Platform	HIGH	Platform Engineer / Toolbox Author	Programmatically generate CI/CD workflow files (GitHub Actions YAML) from MATLAB — project setup tools, release automation, bulk config updates	Primary driver for YAML write support
RG_PYTHON_INTEROP	HIGH	MATLAB Power User	Access Python project metadata from pyproject.toml and MATLAB project metadata from matlab.toml in MATLAB workflows	Primary driver for TOML support
RG_ML_RESEARCHER	HIGH	ML/AI Researcher	Store and load experiment hyperparameter configurations in YAML for reproducible ML workflows	Common in Python-style ML pipelines migrated to MATLAB
RG_CONFIG_MGMT	MEDIUM	Application Developer	Store application settings in human-readable config files	Supports YAML and TOML
RG_CONTAINER	MEDIUM	Platform Engineer	Programmatically generate Docker Compose configurations	YAML with complex nesting
RG_DEVOPS_MATLAB	MEDIUM	DevOps/MATLAB Engineer	Manage YAML configs for MATLAB server deployments (MATLAB Online Server, Production Server, Parallel Server on Kubernetes)	Deployment configs are YAML

Use Cases

Use Case UC_PIPELINE_GENERATION

User Role	Platform Engineer / Toolbox Author (RG_Platform)

CI/CD pipeline configuration needs to be **generated programmatically**. Examples:

- A **project setup tool** that inspects a MATLAB project (which toolboxes it uses, which MATLAB versions it targets) and emits a ready-to-use `.github/workflows/ci.yaml` tailored to that project — driven by MATLAB logic, not hand-authoring
- A **bulk update script** that reads existing workflow files across many repositories, modifies the test matrix (e.g., adds a new MATLAB release), and writes them back — automation at scale rather than repeated manual editing

- A **toolbox packaging tool** that generates CI workflows as part of a release pipeline, filling in project-specific values (name, test paths, artifact names) from MATLAB data structures

MathWorks' `ci-configuration-examples` provides static YAML templates today; this toolbox enables MATLAB code to generate them dynamically. GitHub Actions uses YAML throughout, so YAML write support is the key capability here.

Concrete example — a typical GitHub Actions workflow:

```
name: MATLAB CI

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        matlab-version: [ R2023b, R2024a, R2024b ]
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Setup MATLAB
        uses: matlab-actions/setup-matlab@v2
        with:
          release: ${{ matrix.matlab-version }}
      - name: Run tests
        uses: matlab-actions/run-tests@v2
```

A platform engineer wants to read this file, add `R2025a` to the test matrix, and write it back — entirely from a MATLAB script that manages a collection of repositories. They also want to generate new workflow files from scratch for newly created toolboxes, populating values from MATLAB data.

Current workarounds and their pain:

Today, MATLAB developers generating YAML either switch to Python/shell, or build YAML via string operations:

```
% Today: brittle string assembly
yaml = sprintf('name: %s\njobs:\n  test:\n    runs-on: ubuntu-latest\n',
projectName);
fid = fopen('.github/workflows/ci.yaml', 'w');
fprintf(fid, '%s', yaml);
fclose(fid);
```

This approach has no structure: there is no way to navigate into the document, validate nesting, or safely insert into an existing file. Any modification to an existing workflow means parsing it manually or regenerating it from scratch.

Step	Current Workflow	Pain Point ID(s)
1	Need to generate or update a workflow file from MATLAB logic	PP_NO_YAML_WRITE
2	Construct YAML as a string with `sprintf` or write via Python	PP_FRAGILE
3	Validate output by committing and pushing	PP_NO_LOCAL_GEN

Pain Points:

- **PP_NO_YAML_WRITE:** No native MATLAB way to construct and write structured YAML
- **PP_FRAGILE:** String-based YAML generation is brittle and hard to maintain
- **PP_NO_LOCAL_GEN:** Cannot generate, validate, or preview structured config files locally from within MATLAB

Format features required by this use case:

YAML construct	Where it appears	Requirement
Read YAML	Load existing workflow for modification	R_READ_YAML
Write YAML	Generate and save workflow files	R_WRITE_YAML
Nested mappings	`jobs` → `test` → `strategy` → `matrix`	RY_MAPPINGS
Sequences (flow style)	`branches: [ main, develop ]`, `matlab-version: [ R2023b, ... ]`	RY_SEQUENCES
Sequences (block style)	`steps:` list	RY_SEQUENCES
Sequence of mappings	Each entry under `steps:` is a structured record	RY_SEQ_OF_MAPS
Hyphenated keys	`runs-on`, `pull_request`, `matlab-version`	R_SPECIAL_CHARS
Data round-trip	Read → add release → write back	R_ROUNDTRIP
Formatting control	Generate files that match GitHub Actions conventions	R_FORMAT_OPTIONS

Use Case UC_ML_EXPERIMENTS

User Role **ML/AI Researcher (RG_ML_RESEARCHER)**

Data scientists running MATLAB alongside Python ML tooling use YAML configuration files to define hyperparameters, model architectures, and training schedules. This pattern is standard in the Python ML ecosystem (Hydra, W&B, Kubernetes-style operator configs) and increasingly used by MATLAB researchers working in hybrid environments.

Concrete example — a Hydra-style experiment config:

```yaml
model:
  architecture: resnet50
  pretrained: true
  num_classes: 10

training:
  learning_rate: 0.001
  epochs: 100
  batch_size: 32
  optimizer: adam

data:
  dataset: cifar10
  augmentation: true
  splits:
    train: 0.8
    val: 0.1
    test: 0.1
```

The researcher wants to run a learning rate sweep: load the baseline config, vary `training.learning_rate` across a range, run each experiment, and save the variant config alongside the results so the experiment is reproducible. They also want to compare results across runs by loading the saved configs back into MATLAB.

This workflow is entirely driven by YAML — the config file is the contract between the researcher and the training code. MATLAB has no native way to participate.

Current workarounds and their pain:

The researcher either maintains a Python script to convert YAML to a `.mat` file before running MATLAB, or duplicates config values between a YAML file (for the Python side) and hardcoded MATLAB variables (for the MATLAB side). Neither approach is reproducible: the `.mat` file is not human-readable, and hardcoded values drift out of sync with the YAML.

Step	Current Workflow	Pain Point ID(s)
1	Write hyperparameter config in YAML	PP_NO_YAML
2	Load config from Python, pass to MATLAB	PP_WORKAROUND
3	Run experiment, log results	PP_NOT_REPRODUCIBLE

Pain Points:

- **PP_NO_YAML:** MATLAB cannot natively read YAML experiment configs
- **PP_WORKAROUND:** Requires Python interop or format conversion
- **PP_NOT_REPRODUCIBLE:** Hard to load/save configs programmatically for experiment tracking

Format features required by this use case:

YAML construct	Where it appears	Requirement
Read YAML	Load experiment configs	R_READ_YAML
Write YAML	Save config variants alongside results	R_WRITE_YAML
Nested mappings	`model`, `training`, `data`, `splits` sections	RY_MAPPINGS
Scalar types	Strings, booleans, integers, floats throughout	RY_SCALARS
Data round-trip	Load → modify one value → save variant	R_ROUNDTRIP

Use Case UC_PYPROJECT

User Role	MATLAB Power User (RG_PYTHON_INTEROP)

Python projects that interop with MATLAB describe their metadata in `pyproject.toml`. MATLAB toolbox projects will soon have an analogous `matlab.toml`. MATLAB developers working in these mixed-language environments need to read and write these files from MATLAB — for build scripts, release tools, and dependency management.

Concrete example — a fragment of `pyproject.toml`:

```
[build-system]
requires = ["setuptools>=65.0", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "my-matlab-python-bridge"
version = "0.1.0"
requires-python = ">=3.9"
dependencies = ["numpy>=1.23", "scipy"]

[[project.authors]]
name = "Jane Smith"
email = "jane@example.com"

[[project.authors]]
name = "John Doe"
```

A MATLAB build script needs to extract the project name and version to stamp release artifacts, check the Python version constraint for compatibility, and read the author list to populate release notes — without leaving MATLAB.

Current workarounds and their pain:

MATLAB has no TOML reader. The workaround is to call a Python subprocess to convert the file to JSON, then read the JSON into MATLAB with `jsondecode`. This loses comments, requires Python to be installed and configured, and adds a subprocess call to what should be a simple file read. Alternatively, users write a custom line-by-line parser — fragile and project-specific.

Step	Current Workflow	Pain Point ID(s)
1	Need to read Python package metadata from `pyproject.toml`	PP_NO_TOML
2	Call Python subprocess to convert TOML to JSON	PP_WORKAROUND
3	Read JSON into MATLAB	PP_DATA_LOSS

Pain Points:

- **PP_NO_TOML:** MATLAB has no native TOML support
- **PP_WORKAROUND:** Requires external tools or manual parsing
- **PP_DATA_LOSS:** Conversion loses structure and comments

Format features required by this use case:

TOML construct	Where it appears	Requirement
Read TOML	Load project metadata	R_READ_TOML
Standard tables	`[build-system]`, `[project]`	RT_TABLES
Arrays	`requires = [...]`, `dependencies = [...]`	RT_ARRAYS
Array of tables	`[[project.authors]]` — one entry per author	RT_ARRAY_OF_TABLES
Dotted table names	`[project.urls]`, `[tool.ruff.lint]`	RT_TABLES
Hyphenated keys	`[build-system]`, `build-backend`, `requires-python`	R_SPECIAL_CHARS, RT_HYPHEN_KEYS
Scalar types	Strings, version numbers	RT_SCALARS

Use Case UC_APP_CONFIG

User Role	Application Developer (RG_CONFIG_MGMT)

MATLAB applications that need to be configured by end users or system administrators benefit from a human-readable config file. The user may not be a MATLAB programmer — they just need to adjust settings like server addresses, thresholds, or file paths without touching code.

Concrete example — a MATLAB application config:

```
server:
  host: production.example.com
  port: 8443
  timeout: 30
```

```
  logging:
    level: info
    output: /var/log/myapp.log
    max-size-mb: 100

  processing:
    batch-size: 256
    parallel-workers: 4
    output-dir: /data/results
```

The application reads this file at startup, applies the settings, and the administrator can change them without a MATLAB license or code change. The developer also wants to save a modified config back (e.g., after an auto-tuning run) and have the result be a readable YAML file — not a binary `.mat`.

Current workarounds and their pain:

Application configuration in MATLAB today is typically stored in `.mat` files (binary, unreadable without MATLAB), hardcoded constants in `.m` files (requires code changes and re-deployment), or JSON files (readable but verbose and awkward for config). None of these are human-friendly for administrators to edit directly.

Step	Current Workflow	Pain Point ID(s)
1	Store config as `.mat` file or hardcoded constants	PP_NOT_EDITABLE
2	Load config at runtime	PP_NOT_PORTABLE
3	Share config with non-MATLAB users or system administrators	PP_INTEROP

Pain Points:

- **PP_NOT_EDITABLE:** Users cannot easily modify configuration without MATLAB
- **PP_NOT_PORTABLE:** `.mat` files not readable or editable outside MATLAB
- **PP_INTEROP:** JSON/XML are verbose and unfamiliar for human-authored config

Format features required by this use case:

YAML construct	Where it appears	Requirement
Read YAML	Load config at application startup	R_READ_YAML
Write YAML	Save auto-tuned config back to file	R_WRITE_YAML
Nested mappings	`server`, `logging`, `processing` sections	RY_MAPPINGS
Scalar types	Strings, integers, file paths throughout	RY_SCALARS
Hyphenated keys	`max-size-mb`, `batch-size`, `parallel-workers`, `output-dir`	R_SPECIAL_CHARS
Data round-trip	Load → auto-tune → write readable result	R_ROUNDTRIP

YAML construct	Where it appears	Requirement
Formatting control	Output should be human-readable for administrators	R_FORMAT_OPTIONS

Use Case UC_DEVOPS_INFRA

User Role	DevOps/MATLAB Engineer (RG_DEVOPS_MATLAB)

MathWorks products including MATLAB Online Server, MATLAB Parallel Server, and MATLAB Production Server deploy on Kubernetes using YAML manifests and Helm values files. Platform engineers managing these deployments want to script configuration changes from within MATLAB rather than context-switching to shell or Python.

Concrete example — a Kubernetes deployment fragment:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: matlab-production-server
  namespace: matlab-production
spec:
  replicas: 3
  selector:
    matchLabels:
      app: matlab-server
  template:
    spec:
      containers:
        - name: matlab-server
          image: mathworks/matlab-production-server:R2024b
          resources:
            requests:
              memory: 8Gi
              cpu: "4"
            limits:
              memory: 16Gi
              cpu: "8"
```

A platform engineer wants to scale up the deployment (change `replicas`), update the container image to a new release, and adjust resource limits — all from a MATLAB script that manages a fleet of deployments. They may also generate Helm `values.yaml` files tailored to each deployment environment from a MATLAB data model of their infrastructure.

Current workarounds and their pain:

Platform engineers context-switch to shell or Python to edit these files, then return to MATLAB to run validation or analysis. There is no way to script the full workflow — read the current config, apply a change,

write back, trigger a re-deployment — from a single MATLAB session.

Step	Current Workflow	Pain Point ID(s)
1	Need to modify a Kubernetes deployment YAML	PP_MANUAL_EDIT
2	Context-switch to shell or Python to edit the file	PP_CONTEXT_SWITCH
3	Re-apply and iterate	PP_SLOW_ITERATION

Pain Points:

- **PP_CONTEXT_SWITCH:** Must leave MATLAB to edit deployment configs
- **PP_SLOW_ITERATION:** No programmatic generation of deployment configurations from MATLAB

Format features required by this use case:

YAML construct	Where it appears	Requirement
Read YAML	Load current deployment config	R_READ_YAML
Write YAML	Save modified deployment config	R_WRITE_YAML
Deeply nested mappings	`spec` → `template` → `spec` → `containers` → `resources`	RY_MAPPINGS
Sequence of mappings	`containers:` list — each container is a structured record	RY_SEQ_OF_MAPS
Scalar types	Strings, integers, quoted strings (`"4"`, `"8"`)	RY_SCALARS
Data round-trip	Read → change replica count or image tag → write back	R_ROUNDTRIP

Benchmarks

Benchmarking existing YAML/TOML implementations in Python and MATLAB informed the central design question: **how much fidelity should we target through a read-modify-write cycle?** Three levels of fidelity are possible, each representing a significant jump in implementation complexity:

Level	What it preserves	Implication
Basic reader	Data values only; write support absent or limited	Useful for one-way data extraction; cannot support modify-and-write-back workflows
Data round-trip	All data values and types; comments and formatting may change	Supports all read-modify-write use cases; the de facto standard for config tooling
Perfect round-trip	Comments, whitespace, key ordering, blank lines — structural fidelity	Required only when human-readable formatting or comments must survive automation

The implementations below were examined to calibrate which level to target.

Category: Basic parsers and single-direction readers

matlab-toml (File Exchange #67858)

A MATLAB TOML parser with 609 downloads and 2.0 stars (3 reviews). Read-focused: it parses TOML into MATLAB data structures, but write support is limited and was never a design priority.

Key limitations:

- **Requires unrelated toolboxes** as dependencies, adding installation friction for users who don't already have them
- **Returns `containers.Map`** for TOML tables, which produces awkward workflows: `containers.Map` is a handle class (not value semantics), so copies are not independent; access requires `map('key')` syntax rather than dot notation; and iteration is cumbersome
- **No special-character key support** — keys with hyphens (`build-system`, `x-custom-header`) cannot be represented in `containers.Map` without custom handling
- **No updates since May 2023**, 8 open GitHub issues as of February 2026

Lesson: `containers.Map` is the wrong return type for config data. It leaks implementation details, breaks value semantics, and makes common operations awkward.

tomllib (Python stdlib, Python 3.11+)

Python's built-in TOML reader, added in 3.11. Intentionally read-only by design — the stdlib maintainers explicitly chose not to include a writer, on the grounds that TOML files are usually hand-authored and should not be machine-rewritten.

Key limitations:

- Read-only: no write support at all
- Available only in Python 3.11+; earlier versions need the third-party `tomli` package

Lesson: Read-only is a valid choice for some tools (e.g., a build system that only reads `pyproject.toml`), but does not serve use cases that need to construct or modify configuration programmatically.

Category: Data round-trip implementations

readstruct / writestruct with JSON (MATLAB built-in, R2021b+)

MATLAB's built-in structured data I/O. `readstruct` and `writestruct` support XML and JSON backends. With the JSON backend, they provide data round-trip fidelity for structured config data natively in MATLAB.

Capabilities:

- No external dependencies
- Supports nested structures with dot notation (via struct fields)
- JSON backend provides data type preservation

Limitations:

- JSON only — no YAML or TOML support
- Returns plain `struct`, which has specific ergonomic limitations:
  - **Keys with special characters are silently coerced** to valid MATLAB identifiers (e.g., `key-name` becomes `key_name`), with no way to recover the original key on write-back — the round-trip is structurally lossy for such keys
  - **Arrays of objects become struct arrays**, which return comma-separated lists when indexed across the array (e.g., `s.field` returns a comma-separated list, not an array). Extracting values into a typed array requires an extra step, and writing modified values back typically requires `arrayfun`
- **Single-element arrays lose their arrayness**: a JSON array `[5]` reads back as scalar `5`, not `[5]`

Lesson: `readstruct`/`writestruct` shows that MATLAB users do want structured read/write I/O and that MATLAB's struct type is the obvious baseline return type. The gap is format support (YAML, TOML) and ergonomics for edge cases (special-character keys, array fidelity).

MartinKoch123/yaml (File Exchange #106765)

The most widely-used MATLAB YAML implementation, with 3.4K downloads and 5.0 stars (2 reviews). Actively maintained (v1.6, October 2024). Targets data round-trip fidelity.

Capabilities:

- YAML 1.1 support with bidirectional read/write
- Handles nested structures and arrays
- Actively maintained

Limitations:

- **Requires Java SnakeYAML** — the parser is built on top of the Java SnakeYAML library, which must be on the Java classpath. This works in desktop MATLAB but can fail in headless or deployed environments and introduces a non-MATLAB dependency
- **Returns plain structs** — same limitations as `readstruct`: no dot notation for special-character keys, no structural inspection methods

Lesson: The community has converged on this as the best available YAML option for MATLAB, but the Java dependency is a friction point and plain structs leave ergonomic gaps. A pure-MATLAB implementation returning a more capable object type is a meaningful improvement.

PyYAML (Python)

The dominant Python YAML library with hundreds of millions of monthly downloads. Provides `yaml.safe_load()` for reading and `yaml.dump()` for writing.

Capabilities:

- Full YAML 1.1 support
- `yaml.dump(yaml.safe_load(text))` preserves all data values and types

Round-trip behavior:

- Data values are preserved exactly

- Comments are discarded
- Key ordering may change (depends on Python version and YAML version)
- Formatting and whitespace are normalized to PyYAML's defaults

This is the de facto standard for config file tooling in Python. PyYAML's authors made a deliberate choice not to target comment preservation — the added complexity was not justified for the primary use cases.

Lesson: Data round-trip fidelity is the right target. PyYAML's widespread adoption with this behavior validates that users accept comment loss in programmatic workflows.

Category: Perfect round-trip

ruamel.yaml (Python)

A Python YAML library specifically designed for round-trip preservation. Uses `YAML(typ='rt')` (round-trip) mode to build a comment-preserving AST.

Capabilities:

- Preserves comments, blank lines, key ordering, flow/block style per node
- Near byte-for-byte fidelity through a round-trip

Cost:

- Substantially more complex to use: the round-trip mode returns special `CommentedMap` and `CommentedSeq` objects rather than plain dicts and lists; code that processes the output must handle these types
- The library itself is significantly larger and more complex than PyYAML (the codebase is roughly 5× larger)
- Used primarily in tools where comment preservation is a hard requirement (e.g., `pip-tools`, `conda`)

Lesson: Perfect round-trip is achievable but costly. It is the right choice only when comments must survive automation — for example, tooling that edits config files that humans also maintain by hand and care about keeping their comments intact. For our use cases, data fidelity is sufficient.

Summary of Findings

The benchmarks suggest several conclusions that informed requirements:

1. **Data round-trip fidelity is the right target.** PyYAML's widespread adoption demonstrates that users accept comment loss in programmatic workflows. The added complexity of perfect round-trip (as seen in `ruamel.yaml`) is not justified for config file tooling.

2. **Plain struct is not sufficient as a return type.** Both `readstruct`/`writestruct` and the MATLAB File Exchange submissions return plain structs, and each hits the same walls: special-character keys require silent coercion, struct array indexing is awkward, and there is no method layer for validation or inspection. A value class with dot notation addresses these gaps.

Requirements

Common Requirements

These apply to both YAML and TOML support.

ID	Statement	Pain Point ID(s)	Priority
R_ACCESS	Users can access nested configuration values using natural MATLAB expressions	PP_ERROR_PRONE	MUST HAVE
R_SPECIAL_CHARS	Keys with hyphens and other characters not valid as MATLAB identifiers must be accessible — such keys are ubiquitous in real YAML and TOML files (`runs-on`, `build-system`, `pull-request`)	PP_ERROR_PRONE	MUST HAVE
R_ROUNDTRIP	Data values must be preserved through read → modify → write cycles	PP_DATA_LOSS	MUST HAVE
R_SCALAR_ARRAY	Users can preserve the distinction between a scalar value and a single-element array when that distinction matters for the downstream tool consuming the file	PP_DATA_LOSS	MUST HAVE
R_COPY_INDEPENDENCE	Copying configuration data in MATLAB must produce an independent copy — assigning to a new variable and modifying it must not affect the original	PP_ERROR_PRONE	MUST HAVE
R_DESCRIBE	Users can get a structural overview of what a configuration object contains	—	NICE TO HAVE
R_FORMAT_OPTIONS	Users can control formatting of generated files (indentation, array style, section spacing) to match the conventions of the target tool or project	PP_NOT_EDITABLE	NICE TO HAVE
R_PERFORMANCE	Typical config files (<1MB) must load and save in a reasonable time	—	MUST HAVE
R_NO_DEPS	No external dependencies — pure MATLAB, no Java libraries, MEX files, or toolbox requirements	—	MUST HAVE
R_COMMENT_PRESERVE	Preserve comments through a round-trip	—	OUT OF SCOPE

YAML Requirements

These requirements define what YAML content the toolbox must handle. The goal is to cover the patterns found in real-world CI/CD, container orchestration, and ML experiment config files — not full YAML 1.2 compliance.

ID	Statement	Priority
R_READ_YAML	Users can load YAML configuration files into MATLAB for programmatic access and manipulation	PP_MANUAL_EDIT, PP_NO_YAML
R_WRITE_YAML	Users can write YAML configuration files from MATLAB data	PP_MANUAL_EDIT, PP_FRAGILE
RY_SCALARS	Read and write all common YAML scalar types: strings (quoted and unquoted), booleans (`true`/`false`, `yes`/`no`), integers, floats, and null	MUST HAVE
RY_MAPPINGS	Read and write nested mappings of arbitrary depth	MUST HAVE
RY_SEQUENCES	Read and write sequences in both block style (`- item`) and flow style (`[a, b, c]`)	MUST HAVE
RY_SEQ_OF_MAPS	Read and write sequences of mappings — the pattern used for GitHub Actions `steps:`, Docker Compose `services:`, and Kubernetes containers	MUST HAVE
RY_MIXED_SEQ	Read sequences containing mixed value types (heterogeneous arrays)	MUST HAVE
RY_COMMENTS	Parse and ignore YAML comments; comments need not be preserved on write	MUST HAVE
RY_GITHUB_ACTIONS	Must correctly read and round-trip a GitHub Actions workflow file, including `runs-on`, `pull-request`, `matrix`, and `steps` patterns	MUST HAVE
RY_DOCKER_COMPOSE	Must correctly read and round-trip a Docker Compose file	SHOULD HAVE
RY_KUBERNETES	Must correctly read and round-trip a Kubernetes deployment manifest	SHOULD HAVE

Out of Scope

ID	Statement
RY_ANCHORS	Anchors and aliases (`&anchor`, `*alias`) are not supported
RY_MULTIDOC	Multi-document YAML (files with `---` separators) is not supported
RY_BLOCK_SCALARS	Literal block scalars (`|`) and folded scalars (`>`) are not supported
RY_TAGS	Custom YAML tags (`!!python/object`, etc.) are not supported

TOML Requirements

These requirements define what TOML content the toolbox must handle. The goal is to cover the patterns in `pyproject.toml`, `Cargo.toml`, and the forthcoming `matlab.toml`.

ID	Statement	Priority
R_READ_TOML	Users can load TOML configuration files into MATLAB for programmatic access and manipulation	PP_NO_TOML
R_WRITE_TOML	Users can write TOML configuration files from MATLAB data	PP_NO_TOML
RT_SCALARS	Read and write all TOML scalar types: basic strings, literal strings, integers, floats, booleans, and datetimes	MUST HAVE
RT_TABLES	Read and write standard tables (`[section]`) and dotted-key notation (`a.b = value`)	MUST HAVE
RT_ARRAYS	Read and write arrays, including multiline arrays	MUST HAVE
RT_ARRAY_OF_TABLES	Read and write arrays of tables (`[[section]]`) — used for `[[project.authors]]`, `[[steps]]`, etc.	MUST HAVE
RT_INLINE_TABLES	Read inline tables (`{key = value}`)	MUST HAVE
RT_STRING_TYPES	Read both basic strings (`"..."`, with escape processing) and literal strings (`'...'`, verbatim)	MUST HAVE
RT_HYPHEN_KEYS	Read and write tables with hyphenated names (`[build-system]`, `[build-backend]`)	MUST HAVE
RT_PYPROJECT	Must correctly read and round-trip a `pyproject.toml` including `[build-system]`, `[project]`, `[[project.authors]]`, and `[tool.*]` sections	MUST HAVE
RT_MATLAB_TOML	Must correctly read and write `matlab.toml` as the format is defined	MUST HAVE

Functional Design

Proposed Functional Design: Summary

API Reference Table

YAML

Name	Kind	Description
`readyaml`	Function	Read a YAML file; returns a `YAMLData` object
`writeyaml`	Function	Write a `YAMLData` object (or struct) to a YAML file
`yamldata`	Function	Create a `YAMLData` object — informal constructor
`matlab.io.config.YAMLData`	Class	YAML data container; the type returned by `readyaml`

TOML

Name	Kind	Description
`readtoml`	Function	Read a TOML file; returns a `TOMLData` object
`writetoml`	Function	Write a `TOMLData` object (or struct) to a TOML file
`tomldata`	Function	Create a `TOMLData` object — informal constructor
`matlab.io.config.TOMLData`	Class	TOML data container; the type returned by `readtoml`

Shared base

Name	Kind	Description
`matlab.io.config.ConfigurationData`	Abstract class	Base class for all configuration data types; not instantiated directly; provides the dot notation, key management, and display behavior shared by `YAMLData` and `TOMLData`

Data Class Design

`YAMLData` and `TOMLData` are **value classes** — they behave like MATLAB structs, not objects. Copying produces an independent copy; modifying the copy does not affect the original.

The classes are **struct-like** by design, with a few extensions:

Feature	Behavior
Dot notation access	`config.section.key` — reads and writes nested values in array `config.section` naturally, not comma-separated list
Special character keys	`config.("build-system")` — keys with hyphens, dots, or spaces are fully supported
Key aliases	`config.build_system` also works (auto-generated `makeValidName` alias)
Method calls	Function syntax required: `keys(config)`, `isfield(config, "k")`, `describe(config)` — dot syntax accesses data keys, not methods
Key order	Insertion order preserved on both read and write
Struct compatibility	`struct(config)` converts to a plain struct; writers also accept plain structs

Proposed Design: Details

`readyaml`

Read a YAML file and return a `YAMLData` object.

Syntax

```
data = readyaml(filename)
data = readyaml(filename, SequenceRule=rule)
```

Input Arguments

Argument	Type	Constraints	Description
`filename`	text scalar	Non-empty; file must exist	Path to the YAML file to read

Name-Value Arguments

Name	Type	Values	Default	Description
`SequenceRule`	`string`	`"auto"` | `"cell"`	`"auto"`	Controls how YAML sequences are converted to MATLAB values

`SequenceRule` conversion behavior:

YAML sequence	`"auto"` result	`"cell"` result
`[1, 2, 3]` homogeneous numeric	`[1 2 3]` — double array	`{1, 2, 3}` — cell array
`[a, b, c]` homogeneous text	`["a" "b" "c"]` — string array	`{"a", "b", "c"}` — cell array
`[1, "two", true]` heterogeneous	`{1, "two", true}` — cell array	`{1, "two", true}` — cell array
`[localhost]` (single-element)	`"localhost"` — string scalar; indistinguishable from the non-array form	`{"localhost"}` — 1-element cell; distinguishable from scalar

`SequenceRule` tradeoffs:

	`"auto"`	`"cell"`
Ease of use	High — sequences feel like native MATLAB arrays	Lower — every sequence is a cell array
Round-trip fidelity	Lossy for single-element sequences	Exact structural preservation
Array element access	`config.ports(1)`	`config.ports{1}`
Recommended when	Reading for extraction or modification	Strict structure preservation required

TOML is less affected by this ambiguity. TOML's type system distinguishes arrays from scalars at the syntax level; `SequenceRule` is primarily a YAML concern.

Output Arguments

Argument	Type	Description
`data`	`matlab.io.config.YAMLData`	Configuration data object with dot notation access

Examples

```
% Default: convenient array types
config = readyaml("config.yaml");
config.ports              % [8080, 8443] — double array

% Strict round-trip: all sequences as cell arrays
config = readyaml("config.yaml", SequenceRule="cell");
config.server.allowed_hosts{end+1} = "staging.example.com";
writeyaml(config, "config.yaml");
```

`writeyaml`

Write MATLAB data to a YAML file.

Syntax

```
writeyaml(data)
writeyaml(data, filename)
writeyaml(data, filename, Name=Value, ...)
```

Input Arguments

Argument	Type	Default	Description
`data`	`YAMLData`, `ConfigurationData`, `struct`, `dictionary`, `containers.Map`, cell array, or other MATLAB type	(required)	Data to write to YAML
`filename`	text scalar	`"untitled.yaml"`	Output file path

Name-Value Arguments

Name	Type	Values	Default	Description

Name	Type	Values	Default	Description
`ArrayStyle`	`string`	`"block"` | `"flow"`	`"block"`	Output style for sequences. `"block"` writes each item on its own line with `-` prefix; `"flow"` writes inline as `[a, b, c]`
`NumIndentationSpaces`	`(1,1)` `double`	positive integer	`2`	Number of spaces per indentation level
`SectionSpacing`	`string`	`"loose"` | `"compact"`	`"loose"`	`"loose"` inserts a blank line between top-level keys; `"compact"` omits blank lines
`Precision`	`(1,1)` `double`	positive integer	`6`	Number of significant digits for floating-point values

Output Arguments

None.

Notes

`ArrayStyle` has no `"auto"` mode (unlike `writetoml`). Block style is the YAML convention for config files; use `"flow"` explicitly for compact inline arrays such as short lists.

Examples

```
writeyaml(config, "output.yaml");
writeyaml(config, "output.yaml", ArrayStyle="flow",
NumIndentationSpaces=4);
writeyaml(config, "output.yaml", SectionSpacing="compact", Precision=10);
```

---

yamldata

Create a `YAMLData` object. `yamldata` is the **informal interface** to `matlab.io.config.YAMLData`, following the MATLAB pattern where `figure` creates a `matlab.ui.Figure`. Most users work through `yamldata`; use `matlab.io.config.YAMLData` directly only when subclassing or writing type checks.

Syntax

```
data = yamldata()
data = yamldata(input)
```

Input Arguments

Argument	Type	Description
`input`	`struct` or `dictionary`	Optional. Populate the new object from an existing data structure. When omitted, an empty object is returned.

Output Arguments

Argument	Type	Description
`data`	`matlab.io.config.YAMLData`	New YAMLData object, empty or populated from `input`

Examples

```
% Create empty and populate
config = yamldata();
config.database.host = "localhost";
config.database.port = 5432;

% Create from struct
s.name = "MyApp";
s.version = "1.0";
config = yamldata(s);
```

`matlab.io.config.YAMLData`

YAML configuration data container. This is the type returned by `readyaml` and created by `yamldata`. `YAMLData` is a concrete subclass of `matlab.io.config.ConfigurationData`, which provides dot notation, key management, and display behavior.

Superclasses

Class	Role
`matlab.io.config.ConfigurationData`	Dot notation, key management, display
`matlab.mixin.indexing.RedefinesDot`	Custom dot notation interception
`matlab.mixin.indexing.OverridesPublicDotMethodCall`	Routes dot notation to data keys, not methods
`matlab.mixin.CustomDisplay`	Formatted console display

Reserved Key Name

`YAMLData` has no public properties. One key name is reserved and cannot be used as a configuration key: `xInternal__`.

Dot Notation and Key Aliasing

Dot notation on a `YAMLData` object always accesses data keys, never methods. This is enabled by the `OverridesPublicDotMethodCall` mixin. All methods must be called using **function syntax**:

```
keys(config)       % correct: calls the keys method
config.keys        % NOT a method call — reads the data value under key
"keys"
```

Expression	Behavior
`obj.key`	Read the data value stored under `"key"`
`obj.key = value`	Assign `value` under `"key"`
`obj.("key-name")`	Access a key containing hyphens or other special characters
`obj.alias`	Access via auto-generated `makeValidName` alias (e.g., `obj.build_system` for key `"build-system"`)

Keys with names that are not valid MATLAB identifiers (e.g., `"build-system"`) automatically get a `makeValidName` alias. Both the original key and the alias access the same stored value.

Value Semantics

`YAMLData` is a value class. Assignment creates an independent copy — modifying the copy does not affect the original:

```
copy = original;       % independent copy
copy.field = "new";    % does NOT affect original
```

Nested Object Creation

Assigning to a nested path auto-creates intermediate objects of the same class:

```
config = yamldata;
config.new.section.value = 42;  % creates nested YAMLData objects
automatically
```

Methods

All methods must be called using **function syntax** — `keys(obj)`, not `obj.keys` (see Dot Notation and Key Aliasing above).

Key introspection and manipulation

Method	Description
`keys`	Return all key names in insertion order
`iskey`	Check if a key exists; works element-wise on scalar or array
`remove`	Remove a key; returns updated object
`properties`	Return key names as `cellstr`; used by IDE for tab completion

Struct-compatible aliases

`YAMLData` also implements the standard MATLAB `struct` interface, allowing code written for structs to work without modification:

Method	Equivalent to	Description
`fieldnames`	`keys`	Return key names
`isfield`	`iskey`	Check if a key exists
`rmfield`	`remove`	Remove a key; returns updated object

Display

Method	Description
`show`	Display the data as YAML text in the command window
`describe`	Print or return a structural overview

Conversions

Method	Description
`struct`	Convert to MATLAB struct
`dictionary`	Convert to MATLAB `dictionary`
`map`	Convert to `containers.Map`

`keys`

Return all key names in insertion order.

Syntax

```
k = keys(obj)
```

Input Arguments

Argument	Type	Description
`obj`	`YAMLData` scalar	The configuration data object

Output Arguments

Argument	Type	Description
`k`	`string` array	Key names in insertion order

`iskey`

Check whether a key exists in a `YAMLData` scalar or each element of an array. Useful for filtering arrays of configuration objects.

Syntax

```
tf = iskey(obj, key)
```

Input Arguments

Argument	Type	Description
`obj`	`YAMLData` scalar or array	The configuration data object or array
`key`	text scalar	Key name to check

Output Arguments

Argument	Type	Description
`tf`	`logical` array	Same size as `obj`; `tf(i)` is `true` if `obj(i)` contains `key`

Examples

```
% Filter an array of config objects to those that have "email"
hasEmail = iskey(data.users, "email");
emailUsers = data.users(hasEmail);

% Require all elements to have a key
all(iskey(data.users, "name"))
```

---

remove

Remove a key from the object. Returns the modified object. Because `YAMLData` is a value class, the return value must be captured.

Syntax

```
obj = remove(obj, key)
```

Input Arguments

Argument	Type	Description
`obj`	`YAMLData` scalar	The configuration data object
`key`	text scalar	Key to remove; original keys and aliases both accepted

Output Arguments

Argument	Type	Description
`obj`	`YAMLData`	Updated object with the key removed

Notes

```
config = remove(config, "debug");   % correct: capture the result
remove(config, "debug");            % does nothing — result discarded
```

---

properties

Return key names as a cell array of character vectors. Used by the MATLAB IDE to populate tab completion.

Syntax

```
p = properties(obj)
```

Input Arguments

Argument	Type	Description
`obj`	`YAMLData` scalar	The configuration data object

Output Arguments

Argument	Type	Description
`p`	`cell` of `char`	Key names

`fieldnames`

Return key names as a `string` array. Struct-compatible alias for `keys`.

Syntax

```
names = fieldnames(obj)
```

Input Arguments

Argument	Type	Description
`obj`	`YAMLData` scalar	The configuration data object

Output Arguments

Argument	Type	Description
`names`	`string` array	Key names in insertion order

`isfield`

Check whether a key exists. Struct-compatible alias for `iskey`.

Syntax

```
tf = isfield(obj, key)
```

Input Arguments

Argument	Type	Description
`obj`	`YAMLData` scalar	The configuration data object
`key`	text scalar	Key name to check; original keys and aliases both accepted

Output Arguments

Argument	Type	Description
`tf`	`logical` scalar	`true` if the key exists

`rmfield`

Remove a key from the object. Struct-compatible alias for `remove`; the return value must be captured.

Syntax

```
obj = rmfield(obj, key)
```

Input Arguments

Argument	Type	Description
`obj`	`YAMLData` scalar	The configuration data object
`key`	text scalar	Key to remove; original keys and aliases both accepted

Output Arguments

Argument	Type	Description
`obj`	`YAMLData`	Updated object with the key removed

`show`

Display the data as YAML text in the command window. Useful for viewing deeply nested structures.

Syntax

```
show(obj)
```

Input Arguments

Argument	Type	Description
`obj`	`YAMLData` scalar or array	The object to display

Notes

For scalar objects, displays the YAML serialization of the data. For arrays, displays each element as a YAML list item using `-` block sequence syntax. Falls back to default display if serialization fails.

Examples

```
config = readyaml("config.yaml");
show(config)
```

```
database:
  host: localhost
  port: 5432
  name: mydb
logging:
  level: info
```

```
show(config.database)
```

```
host: localhost
port: 5432
name: mydb
```

---

describe

Print or return a structural overview of the configuration data. When called without an output argument, prints a recursive tree showing all keys, types, and scalar values to the console. When called with an output argument, returns a table for programmatic querying.

Syntax

```
describe(obj)
describe(obj, Depth=N)
info = describe(obj)
info = describe(obj, Depth=N)
```

Input Arguments

Argument	Type	Description
`obj`	`YAMLData` scalar	The configuration data to describe

Name-Value Arguments

Name	Type	Values	Default	Description
`Depth`	`(1,1) double`	positive number	`Inf`	Maximum recursion depth for nested objects

Output Arguments

Argument	Type	Description
`info`	`table`	Only returned when called with an output argument; otherwise output is printed. Columns: `Path` (string), `Type` (string), `Size` (string).

Examples

```
describe(config)
```

```
YAMLData with 2 keys
  database          YAMLData        [1×1, 3 keys]
    host            string          "localhost"
    port            double          5432
    name            string          "mydb"
  logging           YAMLData        [1×1, 2 keys]
    level           string          "info"
    file            string          "app.log"
```

```
describe(config, Depth=1)
```

```
YAMLData with 2 keys
  database          YAMLData        [1×1, 3 keys]
  logging           YAMLData        [1×1, 2 keys]
```

```
info = describe(config);
info(info.Type == "string", :)      % query: all string-valued fields
```

`struct`

Convert the configuration data to a plain MATLAB struct. Keys with special characters are coerced to valid field names via `makeValidName`. Nested `YAMLData` objects are recursively converted to nested structs.

Syntax

```
s = struct(obj)
```

Input Arguments

Argument	Type	Description
`obj`	`YAMLData` scalar or array	The configuration data to convert

Output Arguments

Argument	Type	Description
`s`	`struct` or struct array	Arrays of `YAMLData` produce struct arrays

Notes

Keys with special characters (e.g. `"build-system"`) are coerced to valid field names (e.g. `build_system`). This is one-way — writing the result back to a file will not recover the original key name. For round-trip key fidelity, use the `YAMLData` object directly.

`dictionary`

Convert the configuration data to a MATLAB `dictionary` with string keys and cell values. Nested `YAMLData` objects are recursively converted to nested dictionaries.

Syntax

```
d = dictionary(obj)
```

Input Arguments

Argument	Type	Description
`obj`	`YAMLData` scalar	The configuration data to convert

Output Arguments

Argument	Type	Description
`d`	`dictionary<string, cell>`	Values are wrapped in cells; retrieve with `d{"key"}`

`map`

Convert the configuration data to a `containers.Map`. Provided for compatibility with code that requires `containers.Map`. For new code, prefer `struct` or `dictionary`.

Syntax

```
m = map(obj)
```

Input Arguments

Argument	Type	Description
`obj`	`YAMLData` scalar	The configuration data to convert

Output Arguments

Argument	Type	Description
`m`	`containers.Map`	Key-value map with string keys

`readtoml`

Read a TOML file and return a `TOMLData` object.

Syntax

```
data = readtoml(filename)
data = readtoml(filename, DatetimeType=type)
```

Input Arguments

Argument	Type	Constraints	Description
`filename`	`(1,1) string`	File must exist	Path to the TOML file to read

Name-Value Arguments

Name	Type	Values	Default	Description
`DatetimeType`	`(1,1) string`	`"datetime"` | `"string"`	`"datetime"`	Controls how TOML native datetime values are returned in MATLAB

`DatetimeType` behavior:

`"datetime"`	`"string"`

	`"datetime"`	`"string"`
Return type	MATLAB `datetime`	`string`
Date arithmetic	Available	Not available
Round-trip fidelity	May reformat (timezone normalization, precision)	Exact text preserved
Recommended when	Processing or comparing dates	Preserving exact file representation

TOML has four native datetime subtypes: offset datetime (e.g. `2024-01-15T12:00:00Z`), local datetime (`2024-01-15T12:00:00`), local date (`2024-01-15`), and local time (`12:00:00`). YAML has no native datetime type — dates in YAML are plain strings — so `DatetimeType` applies only to `readtoml`.

Output Arguments

Argument	Type	Description
`data`	`matlab.io.config.TOMLData`	Configuration data object with dot notation access

Examples

```
% Default: TOML dates become MATLAB datetime objects
project = readtoml("pyproject.toml");

% Preserve exact datetime text for round-trip fidelity
project = readtoml("pyproject.toml", DatetimeType="string");
```

`writetoml`

Write MATLAB data to a TOML file.

Syntax

```
writetoml(data)
writetoml(data, filename)
writetoml(data, filename, Name=Value, ...)
```

Input Arguments

Argument	Type	Default	Description

Argument	Type	Default	Description
`data`	`TOMLData`, `ConfigurationData`, `struct`, `dictionary`, or `containers.Map`	(required)	Data to write to TOML
`filename`	`(1,1) string`	`"untitled.toml"`	Output file path

Name-Value Arguments

Name	Type	Values	Default	Description
`ArrayStyle`	`string`	`"auto"` | `"flow"` | `"block"`	`"auto"`	Style for TOML arrays. `"auto"` uses heuristics (flow for short arrays, block for long or complex); `"flow"` forces inline `[1, 2, 3]`; `"block"` forces multiline one-item-per-line
`NumIndentationSpaces`	`(1,1) double`	positive integer	`2`	Number of spaces per indentation level
`SectionSpacing`	`string`	`"loose"` | `"compact"`	`"loose"`	`"loose"` inserts a blank line between top-level tables; `"compact"` omits blank lines
`Precision`	`(1,1) double`	positive integer	`6`	Number of significant digits for floating-point values
`TableStyle`	`string`	`"auto"` | `"inline"` | `"expanded"`	`"auto"`	Style for nested TOML tables. `"auto"` uses heuristics based on size and complexity; `"inline"` forces `{key = value}` syntax; `"expanded"` forces `[section]` header syntax

Name	Type	Values	Default	Description
`TableArrayStyle`	`string`	`"auto"` | `"expanded"` | `"inline"`	`"expanded"`	Style for arrays of tables (`[[section]]`). `"expanded"` writes one `[[section]]` header per record (standard, most readable); `"inline"` writes a compact array of inline tables; `"auto"` uses heuristics
`StringEscapeStyle`	`string`	`"auto"` | `"escaped"` | `"literal"`	`"auto"`	TOML string quoting style. `"escaped"` produces basic strings with `\` escape sequences; `"literal"` produces single-quoted strings where backslashes are verbatim (natural for Windows paths); `"auto"` chooses per-value
`StringLayout`	`string`	`"auto"` | `"singleline"` | `"multiline"`	`"auto"`	Line layout for string values. `"singleline"` forces single-line delimiters; `"multiline"` forces triple-quoted multiline delimiters; `"auto"` chooses based on content

Output Arguments

None.

Notes

- `ArrayStyle` includes `"auto"` (unlike `writeyaml`). Most TOML arrays appear inline in real files; `"auto"` produces natural-looking TOML by default.
- `TableStyle` and `TableArrayStyle` are separate parameters because they control distinct TOML constructs (tables vs. arrays of tables) with different defaults. Heuristic thresholds (e.g., `InlineTableMaxFields`) are not exposed — users choose between `"auto"`, `"expanded"`, and `"inline"`.
- `StringEscapeStyle` and `StringLayout` are orthogonal: escape processing (semantic intent) is independent of line layout (presentation).

Examples

```
writetoml(config, "pyproject.toml");
writetoml(config, "config.toml", ArrayStyle="flow",
SectionSpacing="compact");
writetoml(config, "pyproject.toml", TableArrayStyle="inline");
writetoml(config, "config.toml", StringEscapeStyle="literal");  % natural
for Windows paths
```

---

tomldata

Create a TOMLData object. tomldata is the **informal interface** to matlab.io.config.TOMLData, following the same pattern as yamldata. Most users work through tomldata; use matlab.io.config.TOMLData directly only when subclassing or writing type checks.

Syntax

```
data = tomldata()
data = tomldata(input)
```

Input Arguments

Argument	Type	Description
input	struct or dictionary	Optional. Populate the new object from an existing data structure. When omitted, an empty object is returned.

Output Arguments

Argument	Type	Description
data	matlab.io.config.TOMLData	New TOMLData object, empty or populated from input

Examples

```
config = tomldata();
config.project.name = "my-package";
config.project.version = "1.0.0";
```

---

matlab.io.config.TOMLData

TOML configuration data container. This is the type returned by readtoml and created by tomldata. TOMLData is a concrete subclass of matlab.io.config.ConfigurationData, which provides dot

notation, key management, and display behavior. All properties, methods, and behaviors are identical to `YAMLData`; the only differences are that `SourceFormat` is `"toml"` and nested object creation produces `TOMLData` objects.

Superclasses

Class	Role
`matlab.io.config.ConfigurationData`	Dot notation, key management, display
`matlab.mixin.indexing.RedefinesDot`	Custom dot notation interception
`matlab.mixin.indexing.OverridesPublicDotMethodCall`	Routes dot notation to data keys, not methods
`matlab.mixin.CustomDisplay`	Formatted console display

Reserved Key Name

`TOMLData` has no public properties. One key name is reserved and cannot be used as a configuration key: `xInternal__`.

Dot Notation and Key Aliasing

Identical to `YAMLData`. Dot notation always accesses data keys, never methods; all methods require function syntax. Keys with special characters get `makeValidName` aliases.

```
keys(config)       % correct: calls the keys method
config.keys        % NOT a method call — reads the data value under key
"keys"

config.("build-system")   % original key — always works
config.build_system        % auto-generated alias — also works
```

Value Semantics

Identical to `YAMLData`. `TOMLData` is a value class; assignment creates an independent copy.

Nested Object Creation

Assigning to a nested path auto-creates intermediate `TOMLData` objects:

```
config = tomldata;
config.new.section.value = 42;  % creates nested TOMLData objects
automatically
```

Methods

All methods must be called using **function syntax** — `keys(obj)`, not `obj.keys`.

Key introspection and manipulation

Method	Description
`keys`	Return all key names in insertion order
`iskey`	Check if a key exists; works element-wise on scalar or array
`remove`	Remove a key; returns updated object
`properties`	Return key names as `cellstr`; used by IDE for tab completion

Struct-compatible aliases

`TOMLData` also implements the standard MATLAB `struct` interface, allowing code written for structs to work without modification:

Method	Equivalent to	Description
`fieldnames`	`keys`	Return key names
`isfield`	`iskey`	Check if a key exists
`rmfield`	`remove`	Remove a key; returns updated object

Display

Method	Description
`show`	Display the data as TOML text in the command window
`describe`	Print or return a structural overview

Conversions

Method	Description
`struct`	Convert to MATLAB struct
`dictionary`	Convert to MATLAB `dictionary`
`map`	Convert to `containers.Map`

`keys`

Return all key names in insertion order.

Syntax

```
k = keys(obj)
```

Input Arguments

Argument	Type	Description
`obj`	`TOMLData` scalar	The configuration data object

Output Arguments

Argument	Type	Description
`k`	`string` array	Key names in insertion order

`iskey`

Check whether a key exists in a `TOMLData` scalar or each element of an array. Useful for filtering arrays of configuration objects.

Syntax

```
tf = iskey(obj, key)
```

Input Arguments

Argument	Type	Description
`obj`	`TOMLData` scalar or array	The configuration data object or array
`key`	text scalar	Key name to check

Output Arguments

Argument	Type	Description
`tf`	`logical` array	Same size as `obj`; `tf(i)` is `true` if `obj(i)` contains `key`

Examples

```
% Filter an array of config objects to those that have "email"
hasEmail = iskey(data.authors, "email");
emailAuthors = data.authors(hasEmail);
```

```
% Require all elements to have a key
all(iskey(data.authors, "name"))
```

---

remove

Remove a key from the object. Returns the modified object. Because `TOMLData` is a value class, the return value must be captured.

Syntax

```
obj = remove(obj, key)
```

Input Arguments

Argument	Type	Description
`obj`	`TOMLData` scalar	The configuration data object
`key`	text scalar	Key to remove; original keys and aliases both accepted

Output Arguments

Argument	Type	Description
`obj`	`TOMLData`	Updated object with the key removed

Notes

```
config = remove(config, "deprecated-field");   % correct: capture the
result
remove(config, "deprecated-field");            % does nothing — result
discarded
```

---

properties

Return key names as a cell array of character vectors. Used by the MATLAB IDE to populate tab completion.

Syntax

```
p = properties(obj)
```

Input Arguments

Argument	Type	Description
`obj`	`TOMLData` scalar	The configuration data object

Output Arguments

Argument	Type	Description
`p`	`cell` of `char`	Key names

`fieldnames`

Return key names as a `string` array. Struct-compatible alias for `keys`.

Syntax

```
names = fieldnames(obj)
```

Input Arguments

Argument	Type	Description
`obj`	`TOMLData` scalar	The configuration data object

Output Arguments

Argument	Type	Description
`names`	`string` array	Key names in insertion order

`isfield`

Check whether a key exists. Struct-compatible alias for `iskey`.

Syntax

```
tf = isfield(obj, key)
```

Input Arguments

Argument	Type	Description

Argument	Type	Description
`obj`	`TOMLData` scalar	The configuration data object
`key`	text scalar	Key name to check; original keys and aliases both accepted

Output Arguments

Argument	Type	Description
`tf`	`logical` scalar	`true` if the key exists

`rmfield`

Remove a key from the object. Struct-compatible alias for `remove`; the return value must be captured.

Syntax

```
obj = rmfield(obj, key)
```

Input Arguments

Argument	Type	Description
`obj`	`TOMLData` scalar	The configuration data object
`key`	text scalar	Key to remove; original keys and aliases both accepted

Output Arguments

Argument	Type	Description
`obj`	`TOMLData`	Updated object with the key removed

`show`

Display the data as TOML text in the command window. Useful for viewing the TOML representation of deeply nested structures.

Syntax

```
show(obj)
```

Input Arguments

Argument	Type	Description
obj	TOMLData scalar or array	The object to display

Notes

For scalar objects, displays the TOML serialization of the data. For arrays, displays each element as a TOML array of tables using [[item]] syntax. Falls back to default display if serialization fails.

Examples

```
config = readtoml("pyproject.toml");
show(config)
```

```
[project]
name = "mypackage"
version = "1.0.0"
requires-python = ">=3.9"

[build-system]
requires = ["setuptools"]
build-backend = "setuptools.build_meta"
```

```
show(config.project)
```

```
name = "mypackage"
version = "1.0.0"
requires-python = ">=3.9"
```

---

describe

Print or return a structural overview of the configuration data. When called without an output argument, prints a recursive tree showing all keys, types, and scalar values to the console. When called with an output argument, returns a table for programmatic querying.

Syntax

```
describe(obj)
describe(obj, Depth=N)
```

```
info = describe(obj)
info = describe(obj, Depth=N)
```

Input Arguments

Argument	Type	Description
`obj`	`TOMLData` scalar	The configuration data to describe

Name-Value Arguments

Name	Type	Values	Default	Description
`Depth`	`(1,1) double`	positive number	`Inf`	Maximum recursion depth for nested objects

Output Arguments

Argument	Type	Description
`info`	`table`	Only returned when called with an output argument; otherwise output is printed. Columns: `Path` (string), `Type` (string), `Size` (string).

Examples

```
describe(config)
```

```
TOMLData with 2 keys
  project           TOMLData        [1×1, 3 keys]
    name            string          "mypackage"
    version         string          "1.0.0"
    requires-python string          ">=3.9"
  build-system      TOMLData        [1×1, 2 keys]
    requires        string          [1×1]
    build-backend   string          "setuptools.build_meta"
```

```
describe(config, Depth=1)
```

```
TOMLData with 2 keys
  project           TOMLData        [1×1, 3 keys]
  build-system      TOMLData        [1×1, 2 keys]
```

```
info = describe(config);
info(info.Type == "datetime", :)    % query: all datetime-valued fields
```

`struct`

Convert the configuration data to a plain MATLAB struct. Keys with special characters are coerced to valid field names via `makeValidName`. Nested `TOMLData` objects are recursively converted to nested structs.

Syntax

```
s = struct(obj)
```

Input Arguments

Argument	Type	Description
`obj`	`TOMLData` scalar or array	The configuration data to convert

Output Arguments

Argument	Type	Description
`s`	`struct` or struct array	Arrays of `TOMLData` produce struct arrays

Notes

Keys with special characters (e.g. `"build-system"`) are coerced to valid field names (e.g. `build_system`). This is one-way — writing the result back to a file will not recover the original key name. For round-trip key fidelity, use the `TOMLData` object directly.

`dictionary`

Convert the configuration data to a MATLAB `dictionary` with string keys and cell values. Nested `TOMLData` objects are recursively converted to nested dictionaries.

Syntax

```
d = dictionary(obj)
```

Input Arguments

Argument	Type	Description
`obj`	`TOMLData` scalar	The configuration data to convert

Output Arguments

Argument	Type	Description
`d`	`dictionary<string, cell>`	Values are wrapped in cells; retrieve with `d{"key"}`

`map`

Convert the configuration data to a `containers.Map`. Provided for compatibility with code that requires `containers.Map`. For new code, prefer `struct` or `dictionary`.

Syntax

```
m = map(obj)
```

Input Arguments

Argument	Type	Description
`obj`	`TOMLData` scalar	The configuration data to convert

Output Arguments

Argument	Type	Description
`m`	`containers.Map`	Key-value map with string keys

`matlab.io.config.ConfigurationData`

Abstract base class for all configuration data types. This is an implementation detail — users interact with `YAMLData` and `TOMLData` directly. `ConfigurationData` is relevant when subclassing to create a custom format type, or when writing type checks that apply across all formats: `isa(obj, "matlab.io.config.ConfigurationData")`.

Methods

All methods must be called using **function syntax** — dot notation routes to data keys, not methods. Full documentation for each method is in the `YAMLData` section; the same methods and behaviors apply to all subclasses.

Key introspection and manipulation

Method	Description
`keys`	Return all key names in insertion order
`iskey`	Check if a key exists; works element-wise on scalar or array
`remove`	Remove a key; returns updated object
`properties`	Return key names as `cellstr`; used by IDE for tab completion

Struct-compatible aliases

Method	Equivalent to	Description
`fieldnames`	`keys`	Return key names
`isfield`	`iskey`	Check if a key exists
`rmfield`	`remove`	Remove a key; returns updated object

Display

Method	Description
`show`	Display the data in format-appropriate text (defined per subclass)
`describe`	Print or return a structural overview

Conversions

Method	Description
`struct`	Convert to MATLAB struct
`dictionary`	Convert to MATLAB `dictionary`
`map`	Convert to `containers.Map`

Design Cases

Design Case: Programmatic CI/CD Workflow Generation (yaml)

```
% Generate a GitHub Actions workflow for a MATLAB toolbox project

workflow = yamldata;
workflow.name = "CI";
workflow.on.push.branches = ["main"];
workflow.on.pull__request.branches = ["main"];  % note: key alias for
"pull_request"

% Build the test job
job = yamldata;
job.("runs-on") = "ubuntu-latest";
```

```
job.strategy.matrix.matlab = ["R2022b", "R2023a", "R2024a", "R2025a"];

setupStep = yamldata;
setupStep.name = "Set up MATLAB";
setupStep.uses = "matlab-actions/setup-matlab@v2";
setupStep.with.release = "${{ matrix.matlab }}";

testStep = yamldata;
testStep.name = "Run tests";
testStep.uses = "matlab-actions/run-tests@v2";

job.steps = [setupStep, testStep];
workflow.jobs.test = job;

% Write the generated workflow
writeyaml(workflow, ".github/workflows/ci.yaml");
```

Design Case: ML Experiment Configuration (yaml)

```
% Read hyperparameter config
params = readyaml("experiments/baseline.yaml");

% Inspect structure
describe(params);

% Modify for a learning rate sweep
params.training.learning_rate = 0.001;
params.training.epochs = 100;

% Save variant
writeyaml(params, "experiments/lr_sweep_1.yaml");
```

Design Case: Python Project Metadata (toml)

```
% Read pyproject.toml
project = readtoml("pyproject.toml");

% Access metadata
name    = project.project.name;
version = project.project.version;
authors = project.project.authors;

% Check build dependencies (note: hyphenated key)
deps = project.("build-system").requires;
```

Design Rationale

#	Pros	Priority
1	`read<type>`/`write<type>` naming aligns with modern MATLAB conventions (`readstruct`, `readtable`)	HIGH
2	Dot notation provides natural MATLAB syntax for nested access	HIGH
3	Value class semantics match user expectations for data objects	HIGH
4	Shared base class reduces code duplication across formats	MEDIUM
5	Special character support via `("key-name")` syntax handles real-world files	HIGH
6	Namespace design (`matlab.io.config.*`) follows MathWorks convention; informal wrappers maintain discoverability	HIGH
7	Function-syntax method calls allow config keys to have any name without conflict	HIGH
8	Type validation at assignment provides immediate feedback before a file write is attempted	MEDIUM

#	Cons	Mitigation Plans	Priority
1	Subset parser doesn't support full YAML 1.2 spec	Document limitations clearly; covers 95%+ of config files	MEDIUM
2	Comments not preserved on round-trip	Document as known limitation; common in config parsers	LOW
3	Array/scalar round-trip ambiguity in YAML	`SequenceRule="cell"` option; clear documentation of tradeoff	MEDIUM
4	Function-syntax methods differ from typical MATLAB OOP	Discoverable via tab completion and documentation	LOW

Error Conditions and Edge Cases

#	Condition	Proposed Error / Warning Message
1	File not found	`Error: Unable to open file "filename.yaml". File does not exist.`
2	Invalid YAML syntax	`Error: YAML parse error at line N: <description>`
3	Invalid TOML syntax	`Error: TOML parse error at line N: <description>`
4	Unsupported YAML feature (anchors)	`Warning: YAML anchors and aliases are not supported. Data may be incomplete.`
5	Empty file	Returns empty ConfigurationData object (no error)
6	Key with invalid MATLAB identifier	Automatically creates alias; accessible via `("original-key")`

#	Condition	Proposed Error / Warning Message
7	Assigning unsupported type (e.g., `function_handle`)	`Error: Cannot assign value of type 'function_handle'. Function handles cannot be serialized to a config file.`
8	Assigning a `table`	`Error: Cannot assign table directly. Convert first: struct(yourTable)`
9	Accessing `data.users.name` on a ConfigurationData array	`Error: Cannot access field 'name' on a [1 3] array of YAMLData objects. Index into the array first, e.g., obj(1).name or use: arrayfun(@(x) x.name, obj)`

Alternate Designs Considered

Category A: Return Type Alternatives

A1: Plain MATLAB Structs (readyaml, readtoml return type)

Description: Return plain MATLAB structs from reader functions instead of custom classes.

#	Pros	Priority
1	Familiar to all MATLAB users	HIGH
2	No custom class learning curve	MEDIUM

#	Cons	Priority
1	Cannot handle keys with hyphens/special characters	HIGH
2	Awkward or impossible to round-trip if coercing keys to valid MATLAB identifiers	HIGH
3	Can't customize display to use domain-relevant terminology	MEDIUM

Decision: Rejected due to special character limitation.

A2: containers.Map (readyaml, readtoml return type)

Description: Return `containers.Map` as the primary data structure.

```
config = readyaml("config.yaml");  % Returns containers.Map
config("database")("host")         % Access pattern
```

#	Pros	Priority
1	Native MATLAB type, no custom classes	MEDIUM

#	Pros	Priority
2	Supports any key type	HIGH

#	Cons	Priority
1	Handle class semantics cause confusion (`b = a` shares data)	HIGH
2	Verbose access syntax, not dot notation	HIGH
3	No format-specific metadata	LOW

Decision: Rejected due to handle semantics and verbose syntax.

A3: dictionary (readyaml, readtoml return type)

Description: Return MATLAB's built-in `dictionary` type directly as the data structure

```
config = readyaml("config.yaml");  % Returns dictionary
config("database")("host")         % Access pattern
```

#	Pros	Priority
1	Native MATLAB type with value semantics — copies are independent	HIGH
2	No custom class learning curve	MEDIUM
3	Already required as a minimum-version dependency	LOW

#	Cons	Priority
1	Access syntax is `config("key")` not `config.key` — no dot notation	HIGH
2	Heterogeneous values require `dictionary<string,cell>` — retrieving values needs `{1}` unwrapping	HIGH
3	Nested structures would be `dictionary` of `dictionary` — deeply awkward access patterns	HIGH
4	No custom display, no methods, no format metadata	MEDIUM
5	`dictionary` does not support dot notation overloading	HIGH

Decision: Rejected. While `dictionary` solves the value semantics problem of `containers.Map`, it still cannot provide dot notation access, and the cell-unwrapping required for heterogeneous values makes even simple access verbose. It is the right choice for internal storage (where the class implementation handles the boilerplate), but not as a public return type.

Category B: Class Semantics Alternatives

B1: Handle Class Semantics (ConfigurationData class design)

Description: Use handle class semantics for ConfigurationData objects.

```
config = readyaml("config.yaml");
copy = config;       % Both point to same data
copy.name = "new";   % Modifies original too!
```

#	Pros	Priority
1	Works with older MATLAB versions (pre-R2022b)	MEDIUM
2	Simpler nested modification (no write-back needed internally)	LOW

#	Cons	Priority
1	Handle semantics confuse users: `b = a; b.x = 1` modifies `a`	HIGH
2	Requires explicit `copy()` method for independent copies	MEDIUM
3	Inconsistent with MATLAB data object expectations	HIGH

Decision: Initially implemented, then migrated away. Value semantics provide more intuitive behavior for data objects.

B2: Support Comma-Separated List for Array Field Access (dot notation on ConfigurationData arrays)

Description: Make `arr.name` return comma-separated list like struct arrays when `arr` is a ConfigurationData array.

#	Pros	Priority
1	Matches struct array behavior	MEDIUM
2	Convenient for extracting fields	MEDIUM

#	Cons	Priority
1	ConfigurationData arrays can have heterogeneous keys	HIGH
2	Behavior unpredictable if elements have different keys	HIGH
3	Code relying on this would fail at runtime depending on data	HIGH

Decision: Rejected. Instead, provide helpful error message with workarounds:

```
Cannot access field 'name' on a [1 3] array of TOMLData objects.
Index into the array first, e.g., obj(1).name or use:
  arrayfun(@(x) x.name, obj)
```

Category C: Class Hierarchy Alternatives

C1: Format-Specific Classes Without Shared Base (YAMLData, TOMLData class hierarchy)

Description: Create separate `YAMLData` and `TOMLData` classes without a common `ConfigurationData` base class.

#	Pros	Priority
1	Simpler class hierarchy	LOW
2	Format-specific optimizations possible	LOW

#	Cons	Priority
1	Code duplication across formats	HIGH
2	Inconsistent APIs between formats	MEDIUM
3	Cannot use `isa(obj, 'matlab.io.config.ConfigurationData')` for type checking	MEDIUM

Decision: Rejected. Shared base class reduces duplication and ensures consistent API.

C2: Generic Hash/Map Class (ConfigurationData class scope)

Description: Create a general-purpose dictionary-like class usable beyond configuration files.

#	Cons	Priority
1	Would compete with MATLAB's built-in `dictionary`	HIGH
2	Scope creep beyond configuration file use case	MEDIUM

Decision: Rejected. Too broad; `dictionary` (R2022b+) already serves this purpose.

Category D: Function Naming Alternatives

D1: Format-First Naming (`yamlread`, `yamlwrite`, `tomlread`, `tomlwrite`)

Description: Use `<format>read` / `<format>write` pattern like legacy MATLAB I/O functions.

#	Cons	Priority
1	Inconsistent with modern MATLAB (`readtable`, `readstruct`)	HIGH
2	Legacy pattern is being phased out	MEDIUM
3	`readstruct`/`writestruct` is closest analogue and uses type-first	HIGH

Decision: Rejected. Modern MATLAB uses `read<type>`/`write<type>` pattern.

Category E: Named Argument Alternatives

E1: `ArrayFormat` Instead of `SequenceRule` (readyaml option)

Description: Use `ArrayFormat` parameter name for controlling how YAML sequences are converted.

#	Cons	Priority
1	Confusingly similar to `ArrayStyle` (write option)	HIGH
2	"Format" is ambiguous — could mean output formatting	MEDIUM
3	Doesn't use correct YAML terminology ("sequence")	LOW

Decision: Rejected. `SequenceRule` provides clear distinction from `ArrayStyle` and uses correct YAML terminology. "Rule" suffix matches MATLAB convention (`VariableNamingRule` in `detectImportOptions`).

E2: Exposing Heuristic Thresholds (writetoml options)

Description: Parameters to control automatic behavior thresholds, e.g., `InlineTableMaxFields`.

#	Cons	Priority
1	Overfits API to implementation details	HIGH
2	Heuristics may change; API becomes unstable	HIGH
3	Violates MATLAB philosophy of "useful automatic behavior"	HIGH

Decision: Rejected. Users get `"auto"`, `"expanded"`, or `"inline"` — not threshold tweaking. See `Claude/TOMLWRITE_FORMAT_OPTIONS.md`.

E3: `TableStyle` and `TableArrayStyle` Naming Alternatives (writetoml options)

Candidate	Evaluation
`ArrayOfTablesStyle`	Contains "Of" (prepositions discouraged); verbose
`TableListStyle`	TOML uses "array", not "list"
`TableSequenceStyle`	Not TOML terminology
`RepeatedTableStyle`	Obscure, indirect
`TableArrayStyle`	Clear, concise, TOML-native

Decision: `TableArrayStyle` chosen.

E4: Single `StringStyle` vs `StringEscapeStyle` + `StringLayout` (writetoml options)

Decision: Chose orthogonal parameters:

- `StringEscapeStyle` — Controls escape processing (`"auto"`, `"escaped"`, `"literal"`)
- `StringLayout` — Controls line formatting (`"auto"`, `"singleline"`, `"multiline"`)

Rationale: Separates semantic intent (escape processing) from presentation (layout).

Category F: Terminology Alternatives

F1: "Fields" vs "Keys" vs "Properties" (ConfigurationData method names and display terminology)

Term	Context	Decision
keys	Primary — `keys(obj)`, display, documentation	YAML/TOML specs use "keys"; target users expect it
fields	Alias — `isfield()`, `fieldnames()`, `rmfield()`	Maintains MATLAB struct compatibility
properties	Avoid in display	Conflicts with MATLAB OOP `properties()`

Decision: "Keys" as primary terminology; "fields" supported as aliases for struct compatibility. See `Claude/DESIGN_DECISIONS.md`.

Architectural Design

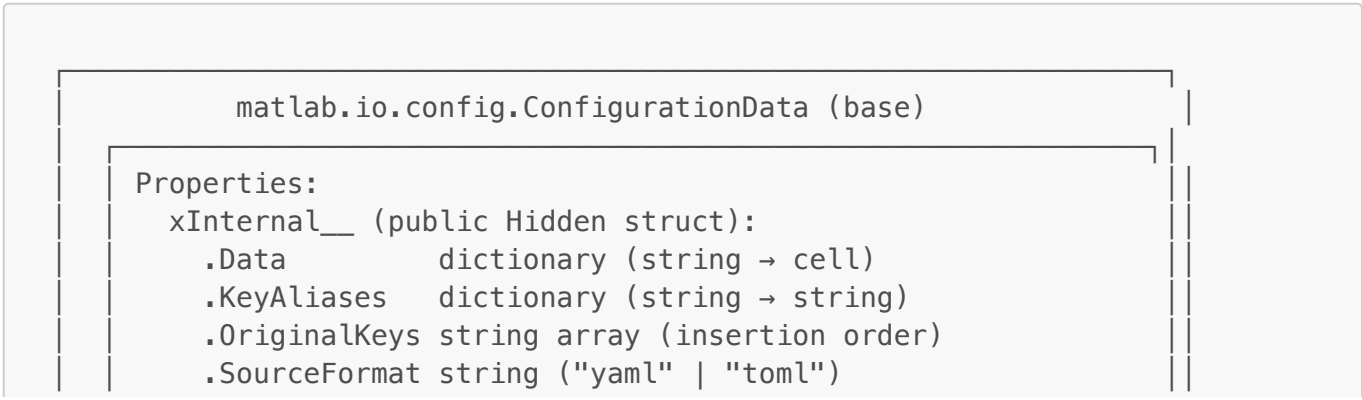
Proposed Architectural Design: Summary

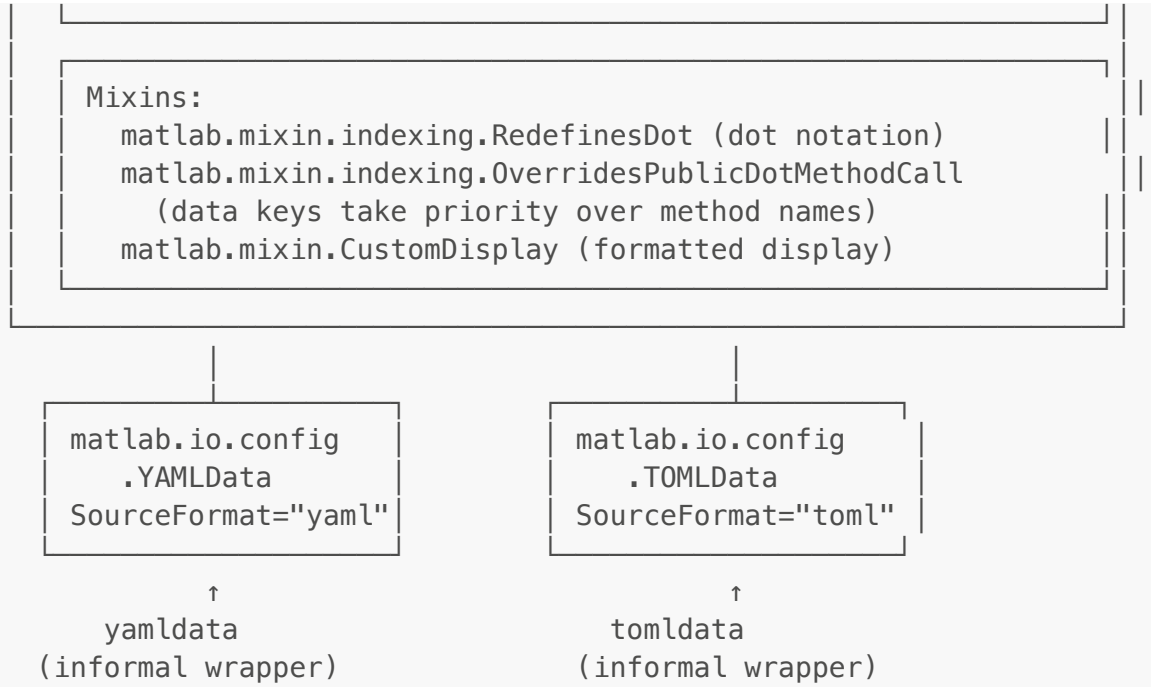
The toolbox uses a **value class hierarchy** with `dictionary`-based storage in the `matlab.io.config` namespace. The `ConfigurationData` base class implements dot notation access via `RedefinesDot` mixin and prioritizes data key access over method names via `OverridesPublicDotMethodCall`. Format-specific subclasses (`YAMLData`, `TOMLData`) handle format identification.

Proposed Design: Details

Architecture Description

Core Components





Value Semantics via dictionary

The critical architectural decision is using `dictionary` (introduced R2022b) instead of `containers.Map`:

Aspect	containers.Map	dictionary
Semantics	Handle (reference)	Value (copy)
Assignment	`b = a` shares data	`b = a` copies data
Modification	`b.x = 1` affects `a`	`b.x = 1` does NOT affect `a`
MATLAB Feel	Confusing	Intuitive

Implementation Pattern:

```
% dictionary requires cell wrapper for heterogeneous values
obj.xInternal__.Data(key) = {value};              % Store
value = obj.xInternal__.Data(key){1};             % Retrieve
obj.xInternal__.Data = remove(obj.xInternal__.Data, key); % Remove
```

Consolidated Internal Storage: xInternal__

All internal state is stored in a single `public Hidden` struct property named `xInternal__`. This consolidation reduces the number of reserved key names from 4 to 1, minimizing conflicts with user configuration keys.

The name `xInternal__` is chosen to be maximally unlikely as a real configuration key:

- `x` prefix: uncommon in config key names
- `__` suffix: Python-style convention signaling internal use

The `public Hidden` visibility is required for MATLAB IDE tab completion: the IDE needs to see the property to enumerate completions, but users shouldn't see it in `disp` output or `properties()`. See `Claude/TAB_COMPLETION_DESIGN.md`.

Informal/Formal Interface Split

Following the `figure` → `matlab.ui.Figure` pattern:

```
% For most users — informal interface, globally available
config = yamldata;                    % creates matlab.io.config.YAMLData
config = tomldata(myStruct);            % creates and populates from struct

% For developers building on top of the toolbox — formal interface
classdef MyAppConfig < matlab.io.config.YAMLData
isa(config, "matlab.io.config.ConfigurationData")  % type check
```

Class constructors are intentionally simple (no conversion logic). Conversion from struct or dictionary happens in the informal wrapper functions, keeping the class hierarchy clean and easy to subclass.

Overview Document

See DESIGN_DECISIONS.md for detailed rationale on:

- Function naming (`read<type>` vs `<type>read`)
- Parameter naming (`SequenceRule` vs `ArrayFormat`)
- Terminology (keys vs fields vs properties)

Non-Functional Requirements

Performance

- **Target:** Config files up to 1MB parsed in <1 second
- **Not optimized for:** Large data files, streaming, or memory-constrained environments
- **Bottlenecks:** String operations in parsers, regex matching in TOML

Compatibility

- **Minimum MATLAB Version:** R2022b (for `dictionary` and `configureDictionary`)
- **Tested Versions:** R2022b, R2023a, R2024a, R2024b
- **Platform:** Windows, macOS, Linux (pure MATLAB)

Architecturally-Significant Design Cases

Case 1: Nested Object Access

The `RedefinesDot` mixin intercepts dot notation and routes through `dotReference` and `dotAssign`:

```
config.database.host = "localhost";

% Internally:
% 1. dotAssign receives indexOp = ["database", "host"]
% 2. Checks if "database" exists, creates YAMLData if not (preserving
class type)
% 3. Recursively assigns "host" = "localhost" to nested object
% 4. Writes modified nested object back (value semantics)
```

Case 2: Special Character Keys

Keys with hyphens create aliases:

```
% YAML: build-system: ...
config.("build-system")  % Primary access — original key
config.build_system      % Alias via makeValidName — also works

% Implementation:
% KeyAliases("build_system") = "build-system"
% Both route to same Data entry
```

Case 3: Array Element Assignment (toml)

The `dotAssign` method handles chained indexing through array elements:

```
% TOML [[users]] array of tables
data.users(2).name = "Suzie";           % Works
data.users(2).permissions.admin = true; % Works (chained)

% dotAssign detects Paren operation in indexOp chain,
% extracts array element, modifies it, writes back
```

Design Rationale

#	Pros	Priority
1	Value semantics eliminate handle class confusion	HIGH
2	`dictionary` is native MATLAB type (R2022b+)	HIGH
3	`RedefinesDot` provides natural syntax	HIGH
4	Base class shares implementation across formats	MEDIUM
5	`matlab.io.config` namespace avoids global namespace pollution	HIGH
6	`OverridesPublicDotMethodCall` allows unrestricted configuration key names	HIGH

#	Cons	Mitigation Plans	Priority
1	Requires R2022b+ (dictionary)	Document requirement clearly	MEDIUM
2	Cell wrapper adds complexity	Encapsulate in base class methods	LOW
3	`obj.field.name` on ConfigurationData array not supported as CSL	Clear error with `arrayfun` workaround	LOW

Alternate Architecture Designs Considered

Alternate Architecture 1: `containers.Map` vs `dictionary` for Internal Storage

Decision: Chose `dictionary` (R2022b+). Its value semantics enable the containing class to have intuitive value behavior without complex copy logic. `containers.Map` is a handle class and would propagate reference semantics to the containing value class.

Alternate Architecture 2: `dynamicprops` Instead of `RedefinesDot`

Description: Use MATLAB's `dynamicprops` mixin to create dynamic properties at runtime.

#	Cons	Priority
1	Cannot handle special characters — `addprop(obj, 'build-system')` fails	HIGH
2	Loses key ordering — `properties()` returns alphabetically sorted	HIGH
3	Heterogeneous arrays fail — `arr.email` errors if any element lacks `email`	HIGH

Decision: Rejected. Special characters are essential for TOML/YAML (kebab-case keys like `build-system` are ubiquitous). See `Claude/DYNAMICPROPS_ANALYSIS.md`.

Alternate Architecture 3: Struct with Metadata Wrapper

Description: Return structs with a thin wrapper for metadata only.

Decision: Rejected due to special character limitation.

Alternate Architecture 4: Override `subsasgn` for Array Indexing

Description: To fix array indexing, override `subsasgn` instead of enhancing `dotAssign`.

#	Cons	Priority
1	`RedefinesDot` explicitly forbids `subsasgn` override	HIGH
2	Would conflict with mixin's internal implementation	HIGH

Decision: Rejected. Fixed by enhancing `dotAssign` to handle `Paren` type in indexOp chain. See `Claude/ARRAY_INDEXING_LIMITATIONS.md`.

Test Strategy & Testability

Test Strategy

Unit Tests

Test File	Coverage
`tests/yamltest.m`	YAML read, write, round-trip, edge cases
`tests/tomltest.m`	TOML read, write, round-trip, arrays of tables
`tests/subsasgnTest.m`	Array element assignment patterns

Test Categories

1. **Basic Reading Tests**
   - Simple key-value pairs
   - Nested structures
   - Arrays (block and flow style)
   - Special characters in keys
2. **Basic Writing Tests**
   - Simple data output
   - Nested structures
   - Array formatting options
   - Section spacing
3. **Round-Trip Tests**
   - Read → Write → Read produces identical data
   - `SequenceRule="auto"` and `SequenceRule="cell"` round-trip behavior
   - Real-world files: GitHub Actions, Docker Compose, Kubernetes, pyproject.toml
   - Complex nesting and arrays
4. **Edge Cases**
   - Empty files
   - Comments (should be ignored)
   - Quoted strings
   - Boolean variations (`true`, `yes`, `on`)
   - Null values
5. **Value Semantics Tests**

- Assignment creates independent copy
- Modification doesn't affect original
- Nested modification behavior

6. **Array Indexing Tests**
   - `data.users(2).name = value` assignment
   - `data.users(2).permissions.admin = true` chained assignment
   - Array element read access

Sample Test Files

Located in `tests/SampleFiles/`:

- `server_config.yaml`
- `arrays_config.yaml`
- `simple-github-actions.yaml`
- `simple-docker-compose.yaml`
- `kubernetes-service.yaml`
- `github-actions-ci.yaml`
- `kubernetes-deployment.yaml`

Test Execution

```
% Run all YAML tests
results = runtests("tests/yamltest.m");

% Run specific test
results = runtests("yamltest/testRoundtripSimpleDockerCompose");

% Run all tests in project
results = runtests(pwd, IncludeSubfolders=true);
```

Testability Features

- **Pure MATLAB:** No external dependencies simplify test environment
- **Sample Files:** Curated collection of real-world config files
- **Temporary Files:** Tests use `TemporaryFolderFixture` for isolation
- **Round-Trip Pattern:** Standard verification approach

Documentation Notes

Existing Documentation

Document	Location	Purpose
README.md	Project root	User-facing quick start and overview

Document	Location	Purpose
GettingStarted.mlx	Project root	Interactive MATLAB tutorial
examples/*.m	examples/	Runnable example scripts
Claude/*.md	Claude/	Development documentation and design decisions

Documentation Deliverables

- [ ] Function reference pages (readyaml, writeyaml, readtoml, writetoml)
- [ ] Class reference page (ConfigurationData, YAMLData, TOMLData) with `matlab.io.config` namespace
- [ ] Note on informal vs formal interface (`yamldata` vs `matlab.io.config.YAMLData`)
- [ ] Round-trip behavior guide: SequenceRule tradeoffs with concrete examples
- [ ] "Working with YAML Data" example
- [ ] "Working with TOML Data" example
- [ ] Limitations and troubleshooting guide

Key Documentation Points

1. **Minimum MATLAB Version:** R2022b (for dictionary support)
2. **Subset Parser:** Not full YAML 1.2 or TOML 1.0 compliance
3. **Value Semantics:** Assignment creates copies (unlike handle classes)
4. **Special Characters:** Use `("key-name")` syntax for keys with hyphens
5. **Round-Trip:** Data preserved, but comments and formatting may change; single-element arrays may become scalars with default `SequenceRule="auto"`
6. **Method Calls:** Use function syntax — `keys(config)`, not `config.keys`
7. **Reserved Key:** `xInternal__` cannot be used as a configuration key